

## 1.1 User Authentication Screens

Students must document:

- Registration page screenshot

The screenshot shows the 'Create an Account' page for BookStore. The page has a blue header with the BookStore logo, 'Home' and 'Books' links, and 'Login' and 'Register' buttons. The main content area is white and contains a registration form. The form has the following fields: 'First Name' (Harsha), 'Last Name' (Kumaraharsha), 'Username' (Harsha), 'Email Address' (harsha@gmail.com), 'Password' (masked with dots), and 'Confirm Password' (masked with dots). Below the form is a blue button labeled 'Create Account' and a link 'Already have an account? Sign in'.

- Login page screenshot

The screenshot shows the 'Welcome Back' login page for BookStore. The page has a blue header with the BookStore logo, 'Home' and 'Books' links, and 'Login' and 'Register' buttons. The main content area is white and contains a login form. The form has the following fields: 'Email Address' (harsha@gmail.com) and 'Password' (masked with dots). Below the form is a blue button labeled 'Sign In' and a link 'Don't have an account? Sign up'. There are also checkboxes for 'Remember me' and a link 'Forgot password?'. The footer is black and contains the BookStore logo, contact information, and links to 'Company', 'Our Service', and 'Corporate' sections.

- Associated HTML/CSS/JavaScript code screenshots for each page

The top screenshot shows the code for the Register page. It includes imports for useState, useRouter, Link, FiUser, FiMail, FiLock, FiUserPlus, and useAuth. The Register function uses useState to manage form data and errors, and useAuth to handle the registration logic. The validate function checks for required fields and minimum lengths.

```

1 'use client';
2
3 import { useState } from 'react';
4 import { useRouter } from 'next/navigation';
5 import Link from 'next/link';
6 import { FiUser, FiMail, FiLock, FiUserPlus } from 'react-icons/fi';
7 import { useAuth } from '../context/AuthContext';
8
9 export default function Register() {
10   const [formData, setFormData] = useState({
11     username: '',
12     email: '',
13     password: '',
14     confirmPassword: '',
15     firstName: '',
16     lastName: ''
17   });
18   const [errors, setErrors] = useState<Record<string, string>>({});
19   const { register, loading } = useAuth();
20   const router = useRouter();
21
22   const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
23     const { name, value } = e.target;
24     setFormData({ ...formData, [name]: value });
25   };
26
27   const validate = () => {
28     const newErrors: Record<string, string> = {};
29
30     if (!formData.username) {
31       newErrors.username = 'Username is required';
32     } else if (formData.username.length < 3) {
33       newErrors.username = 'Username must be at least 3 characters';
34     }
35
36     if (!formData.email) {
37       newErrors.email = 'Email is required';

```

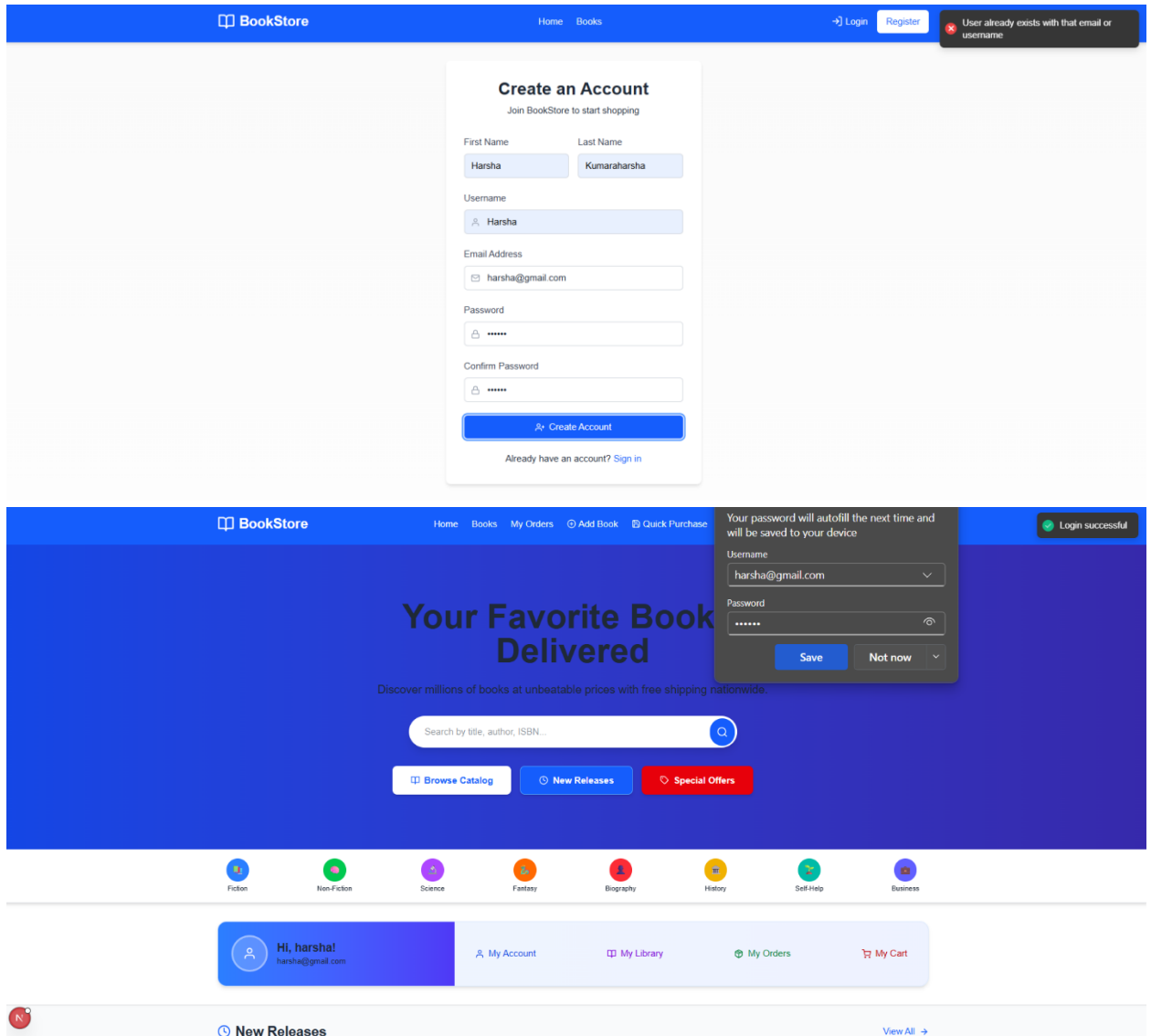
The bottom screenshot shows the code for the Login page. It includes imports for useState, useRouter, Link, FiMail, FiLock, FiLogin, and useAuth. The Login function uses useState to manage form data and errors, and useAuth to handle the login logic. The validate function checks for required fields and email/password format.

```

1 'use client';
2
3 import { useState } from 'react';
4 import { useRouter } from 'next/navigation';
5 import Link from 'next/link';
6 import { FiMail, FiLock, FiLogin } from 'react-icons/fi';
7 import { useAuth } from '../context/AuthContext';
8
9 export default function Login() {
10   const [email, setEmail] = useState('');
11   const [password, setPassword] = useState('');
12   const [errors, setErrors] = useState<{ email?: string; password?: string }>({});
13   const { login, loading } = useAuth();
14   const router = useRouter();
15
16   const validate = () => {
17     const newErrors: { email?: string; password?: string } = {};
18
19     if (!email) {
20       newErrors.email = 'Email is required';
21     } else if (!/^[a-zA-Z0-9@.\+\-\/\%]+$/i.test(email)) {
22       newErrors.email = 'Email is invalid';
23     }
24
25     if (!password) {
26       newErrors.password = 'Password is required';
27     }
28
29     setErrors(newErrors);
30     return Object.keys(newErrors).length === 0;
31   };
32
33   const handleSubmit = async (e: React.FormEvent) => {
34     e.preventDefault();
35
36     if (!validate()) return;

```

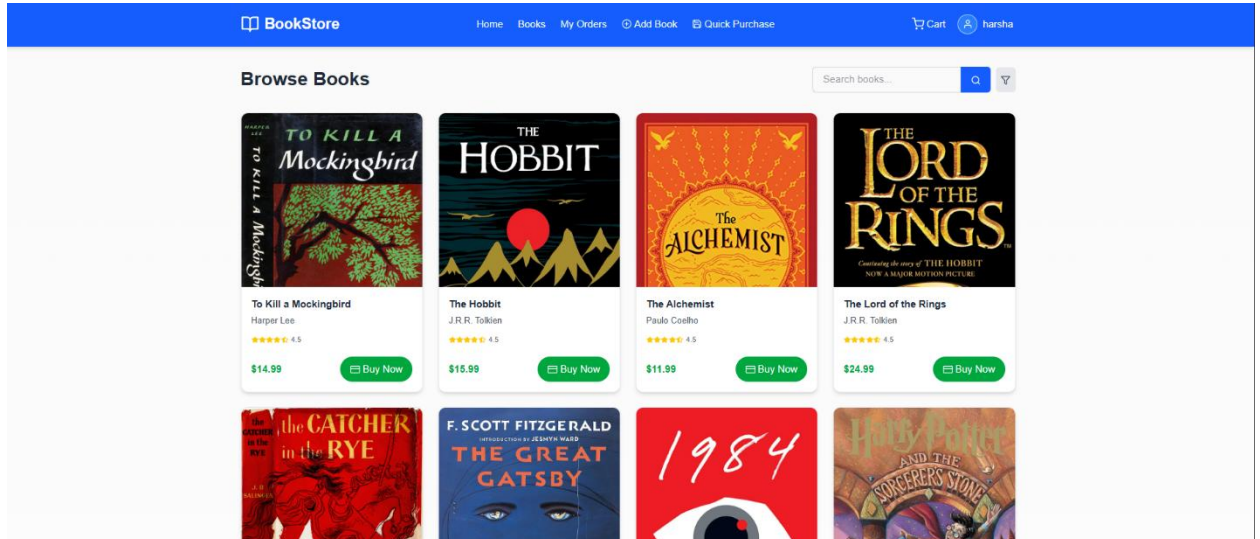
- Authentication success/error messages screenshots



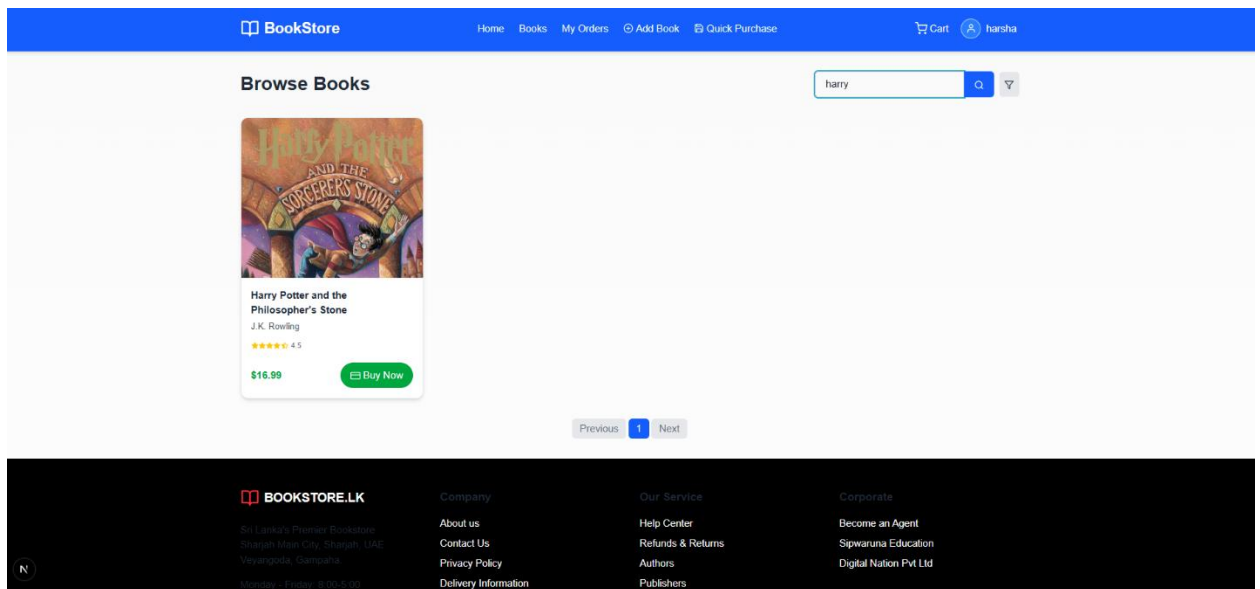
## 1.2 Book Listing and Search

Required documentation:

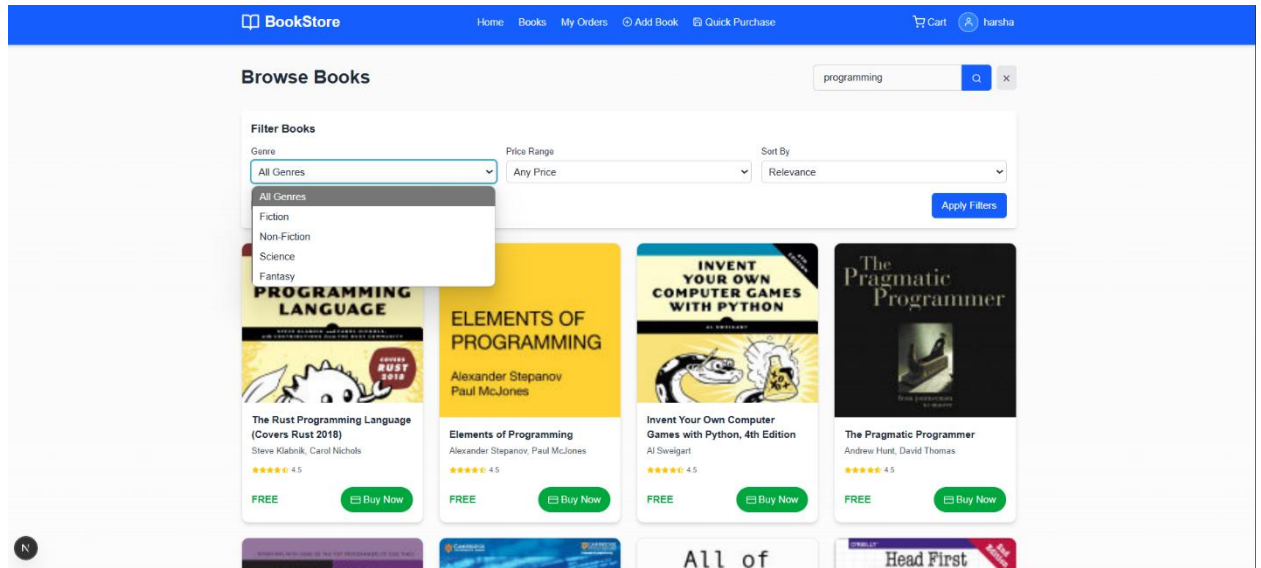
- Main book listing page screenshot



- Search functionality interface screenshot



- Filter/Sort options screenshot



- Book details view screenshot



- Code screenshots for API integration

The screenshot shows the VS Code editor with the file explorer on the left, the source code in the center, and the terminal at the bottom. The file explorer shows the project structure with 'frontend' > 'app' > 'books' > 'page.tsx' selected. The source code displays the `searchGoogleBooks` function, which uses `useEffect` to fetch data and a custom `searchGoogleBooks` function to search the Google Books API. The terminal shows the output of running the application, indicating it is connected to MongoDB and running on port 5000.

```

9 export default function Books() {
25   useEffect(() => {
26     const fetchData = async () => {
51     };
52   });
53   fetchData();
54   }, [currentPage, searchQuery, freeEbooksOnly]);
55
56   // Add a new function to search Google Books API directly
57   const searchGoogleBooks = async () => {
58     setloading(true);
59     setError(null);
60
61     // Use default search term if empty
62     const searchTerm = searchQuery || 'programming';
63
64     try {
65       // Direct call to Google Books API for free eBooks
66       const googleBooksApiKey = 'AIzaSyBt0N5P5E1jUR8mKAVEVOJm_uFcPcBY';
67       const filter = freeEbooksOnly ? 'free-ebooks' : '';
68       let url = `https://www.googleapis.com/books/v1/volumes?q=${encodeURIComponent(searchTerm)}`;
69
70       if (filter) {
71         url += `&filter=${filter}`;
72       }
73
74       url += `&startIndex=${(currentPage - 1) * itemsPerPage}&maxResults=${itemsPerPage}&key=${googleBooksApiKey}`;
75
76       const response = await fetch(url);
77       const data = await response.json();
78       setBooks(data.items);
79     } catch (error) {
80       setError(error.message);
81     }
82   };
83
84   return (
85     <div className="container mx-auto px-4 py-8">
86       <h1 className="text-3xl font-bold mb-4 md:mb-0">Browse Books</h1>
87
88       <div className="flex items-center space-x-2 w-full md:w-auto">
89         <form onSubmit={handleSearch} className="flex flex-grow">
90           <input
91             type="text"
92             value={searchQuery}
93             onChange={(e) => setSearchQuery(e.target.value)}
94             placeholder="Search books..."
95             className="px-4 py-2 border border-gray-300 rounded-l-md focus:outline-none focus:ring-2 focus:ring-blue-500 w-full"
96           />
97           <button
98             type="submit"
99             className="bg-blue-600 text-white px-4 py-2 rounded-r-md hover:bg-blue-700 transition-colors"
100           >
101             <FiSearch />
102           </button>
103         </form>
104
105         <div className="flex items-center space-x-2">
106           <button
107             onClick={() => setIsFilterOpen(!isFilterOpen)}
108             className="bg-gray-200 text-gray-700 p-2 rounded-md hover:bg-gray-300 transition-colors"
109             aria-label="Filter"
110           >
111             <FiFilter />
112           </button>
113         </div>
114       </div>
115
116       <div className="flex flex-wrap">
117         {books.map((book) => (
118           <div key={book.id} className="flex flex-wrap justify-between items-start md:items-center mb-8">
119             <div className="flex flex-col md:flex-row justify-between items-start md:items-center mb-8">
120               <div>
121                 <h2>{book.title}</h2>
122                 <p>{book.author}</p>
123               </div>
124               <div>
125                 <img alt={book.image}</img>
126               </div>
127             </div>
128             <div>
129               <p>{book.description}</p>
130             </div>
131           </div>
132         ))}
133       </div>
134     </div>
135   );
136 }

```

- Search implementation code screenshots

The screenshot shows the VS Code editor with the file explorer on the left, the source code in the center, and the terminal at the bottom. The file explorer shows the project structure with 'frontend' > 'app' > 'books' > 'page.tsx' selected. The source code displays the `Books` component, which renders a search form and a filter button. The terminal shows the output of running the application, indicating it is connected to MongoDB and running on port 5000.

```

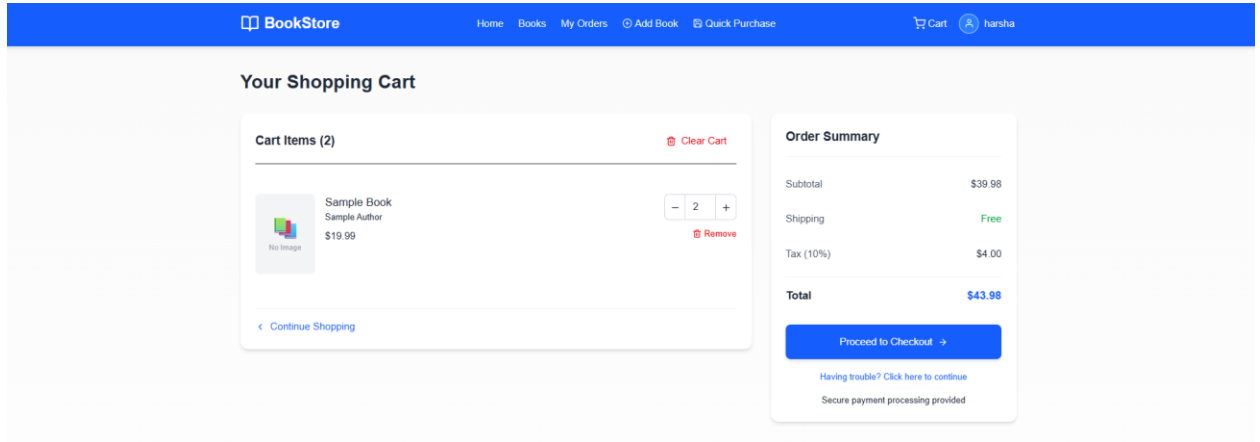
9 export default function Books() {
153   return (
154     <div className="container mx-auto px-4 py-8">
155       <div className="flex flex-col md:flex-row justify-between items-start md:items-center mb-8">
156         <h1 className="text-3xl font-bold mb-4 md:mb-0">Browse Books</h1>
157
158         <div className="flex items-center space-x-2 w-full md:w-auto">
159           <form onSubmit={handleSearch} className="flex flex-grow">
160             <input
161               type="text"
162               value={searchQuery}
163               onChange={(e) => setSearchQuery(e.target.value)}
164               placeholder="Search books..."
165               className="px-4 py-2 border border-gray-300 rounded-l-md focus:outline-none focus:ring-2 focus:ring-blue-500 w-full"
166             />
167             <button
168               type="submit"
169               className="bg-blue-600 text-white px-4 py-2 rounded-r-md hover:bg-blue-700 transition-colors"
170             >
171               <FiSearch />
172             </button>
173           </form>
174
175           <div className="flex items-center space-x-2">
176             <button
177               onClick={() => setIsFilterOpen(!isFilterOpen)}
178               className="bg-gray-200 text-gray-700 p-2 rounded-md hover:bg-gray-300 transition-colors"
179               aria-label="Filter"
180             >
181               <FiFilter />
182             </button>
183           </div>
184         </div>
185
186         <div className="flex flex-wrap">
187           {books.map((book) => (
188             <div key={book.id} className="flex flex-wrap justify-between items-start md:items-center mb-8">
189               <div className="flex flex-col md:flex-row justify-between items-start md:items-center mb-8">
190                 <div>
191                   <h2>{book.title}</h2>
192                   <p>{book.author}</p>
193                 </div>
194                 <div>
195                   <img alt={book.image}</img>
196                 </div>
197               </div>
198               <div>
199                 <p>{book.description}</p>
200               </div>
201             </div>
202           ))}
203         </div>
204       </div>
205     </div>
206   );
207 }

```

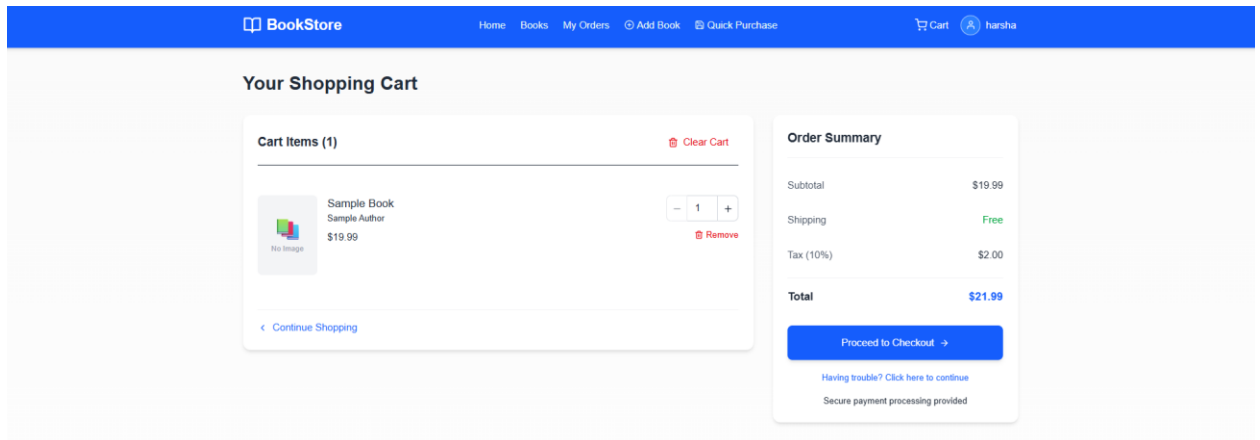
## 1.3 Shopping Cart

Document the following:

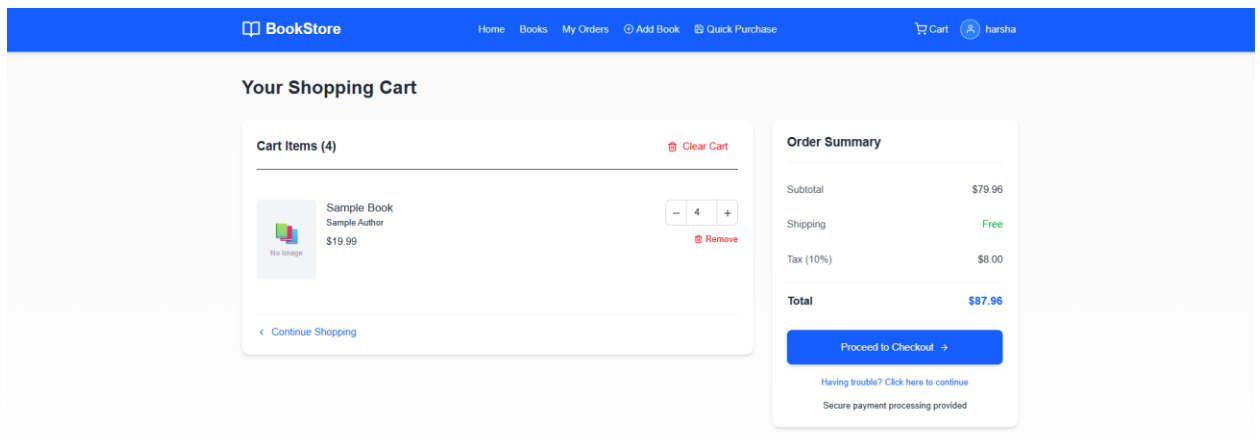
- Cart view screenshot



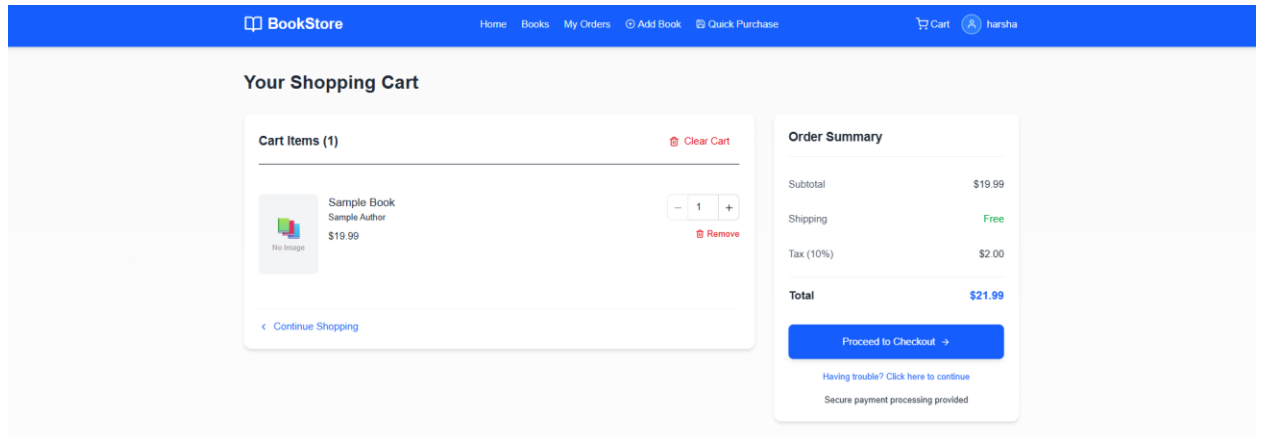
- Add to cart functionality screenshot



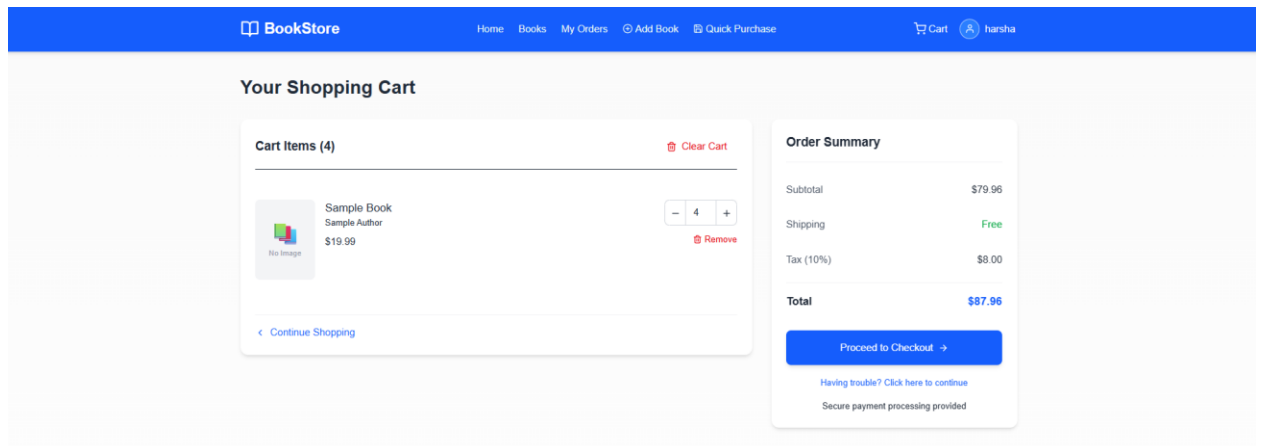
- Update quantity interface screenshot



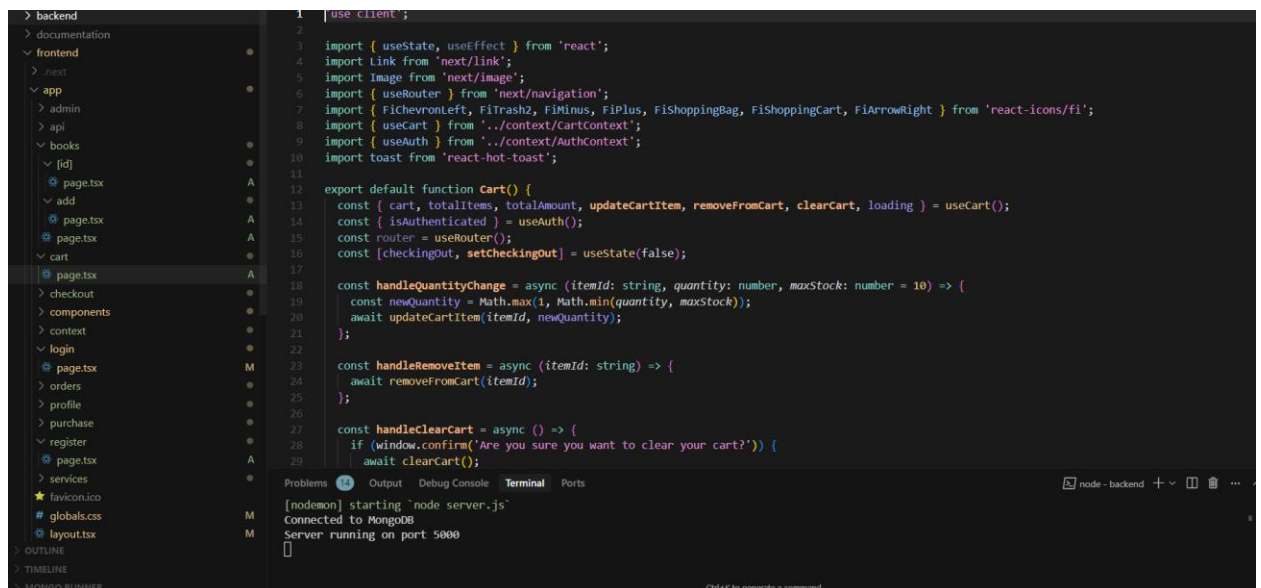
- Remove items interface screenshot



- Cart total calculation screenshot



- Related code screenshots for cart management





## 1.4 Checkout Process

Include:

- Checkout form screenshot

The screenshot shows the 'Complete Your Purchase' page in the 'BookStore' application. The navigation bar at the top includes 'Home', 'Books', 'My Orders', 'Add Book', 'Quick Purchase', a shopping cart icon, and the user name 'harsha'. The page title is 'Complete Your Purchase'. A progress bar at the top indicates three steps: 'Review Order' (active), 'Payment', and 'Confirmation'. The 'Review Order' section is divided into two columns. The left column, 'Book Details', shows a book titled 'Sample Book' by 'Sample Author' for \$19.99, categorized as 'Fiction'. It includes a quantity selector set to 1 and a description: 'This is a sample book description for development mode.' A 'Magic Purchase' button is at the bottom. The right column, 'Order Summary', lists the book, quantity, price per unit, subtotal, shipping, tax, and a total of \$21.59. A 'Proceed to Payment' button is at the bottom.

| Book Details                                                         |                   |
|----------------------------------------------------------------------|-------------------|
| Sample Book                                                          | Sample Author     |
| Price: \$19.99                                                       | Category: Fiction |
| Quantity: 1                                                          |                   |
| Description: This is a sample book description for development mode. |                   |
| <a href="#">Magic Purchase</a>                                       |                   |

| Order Summary                      |             |
|------------------------------------|-------------|
| Book                               | Sample Book |
| Quantity                           | 1           |
| Price per unit                     | \$19.99     |
| Subtotal                           | \$19.99     |
| Shipping                           | \$0.00      |
| Tax                                | \$1.60      |
| Total                              | \$21.59     |
| <a href="#">Proceed to Payment</a> |             |

- Payment interface screenshot

The screenshot shows the 'Complete Your Purchase' page in the 'BookStore' application, specifically the 'Payment' step. The navigation bar at the top includes 'Home', 'Books', 'My Orders', 'Add Book', 'Quick Purchase', a shopping cart icon, and the user name 'admin'. The page title is 'Complete Your Purchase'. A progress bar at the top indicates three steps: 'Review Order', 'Payment' (active), and 'Confirmation'. The 'Payment' section is divided into two columns. The left column, 'Book Details', is identical to the previous screenshot. The right column, 'Payment Information', contains fields for 'Card Number' (1234 5678 9012 3456), 'Cardholder Name' (John Doe), 'Expiry Date' (MM/YY), and 'CVV' (123). There is a checkbox for 'Save this card for future purchases' and two buttons: 'Back to Review' and 'Complete Purchase'. A message at the bottom states 'Your card will be charged \$21.59'.

| Payment Information                                          |                                   |
|--------------------------------------------------------------|-----------------------------------|
| Card Number                                                  | 1234 5678 9012 3456               |
| Cardholder Name                                              | John Doe                          |
| Expiry Date                                                  | MM/YY                             |
| CVV                                                          | 123                               |
| <input type="checkbox"/> Save this card for future purchases |                                   |
| <a href="#">Back to Review</a>                               | <a href="#">Complete Purchase</a> |
| Your card will be charged \$21.59                            |                                   |

BookStore

HomeBooksMy OrdersAdd BookQuick Purchase

Cartadmin

Complete Your Purchase

Review Order

Payment

Confirmation

Book Details

Sample Book

Sample Author

Price: \$19.99

Category: Fiction

+

1

+

Description

This is a sample book description for development mode.

Magic Purchase

Payment Information

Card Number

1234 5214 4512 5632

Cardholder Name

Harsha Kumara

Expiry Date

07/26

CVV

123

☒ Save this card for future purchases

Back to Review

Complete Purchase

Your card will be charged \$21.59

• Order confirmation page screenshot

BookStore

HomeBooksMy OrdersAdd BookQuick Purchase

Cartadmin

Back to Orders

Order #ORD-152178

Placed on May 6, 2025 at 10:35 PM

Total: \$21.59

processing

Download Invoice

Order Status

Order Delivered

Pending

Order Shipped

Pending

Order Processing

May 6, 2025 at 10:35 PM

Order Placed

May 6, 2025 at 10:35 PM

Order Items

Book Title

Unknown Author

1 x \$19.99

\$19.99

- Implementation code screenshots

```

12 interface DirectPurchaseItem {
13   book: any;
14   bookId: string;
15   quantity: number;
16   price: number;
17 }
18
19 export default function checkout() {
20   const router = useRouter();
21   const searchParams = useSearchParams();
22   const { cart, refreshCart, totalItems } = useCart();
23   const { user, isAuthenticated } = useAuth();
24
25   // Handle direct purchase parameters
26   const bookId = searchParams?.get('bookId');
27   const quantity = parseInt(searchParams?.get('quantity') || '1', 10);
28
29   const [directPurchaseItem, setDirectPurchaseItem] = useState<DirectPurchaseItem | null>(null);
30   const [isDirectPurchase, setIsDirectPurchase] = useState(false);
31   const [step, setStep] = useState(1);
32   const [loading, setloading] = useState(false);
33   const [loadingBook, setloadingBook] = useState(false);
34
35   const [shippingInfo, setshippingInfo] = useState({
36     name: '',
37     address: {
38       street: '',
39       city: '',
40     },
41   });
42
43   [nodemon] starting `node server.js`
44   Connected to MongoDB
45   Server running on port 5000

```

- Order processing code screenshots

```

6 import { FiArrowLeft, FiCreditCard, FiLock, FiShield, FiCheckCircle, FiAlertCircle } from 'react-icons/fi';
7 import { useAuth } from '../context/AuthContext';
8 import { ordersAPI, cartAPI } from '../services/api';
9 import toast from 'react-hot-toast';
10
11 export default function PaymentPage() {
12   const router = useRouter();
13   const searchParams = useSearchParams();
14   const { isAuthenticated, loading, user } = useAuth();
15
16   const [paymentInfo, setPaymentInfo] = useState({
17     method: 'credit_card',
18     cardNumber: '',
19     cardholderName: '',
20     expiryDate: '',
21     cvv: '',
22     saveCard: false
23   });
24
25   const [orderDetails, setOrderDetails] = useState({
26     orderId: '',
27     amount: 0,
28     tax: 0,
29     total: 0
30   });
31
32   const [errors, setErrors] = useState<Record<string, string>>({});
33   const [processing, setProcessing] = useState(false);
34   const [status, setStatus] = useState<'idle' | 'processing' | 'success' | 'error'>('idle');
35
36   [nodemon] starting `node server.js`
37   Connected to MongoDB
38   Server running on port 5000

```

## 1.5 User Profile

Document:

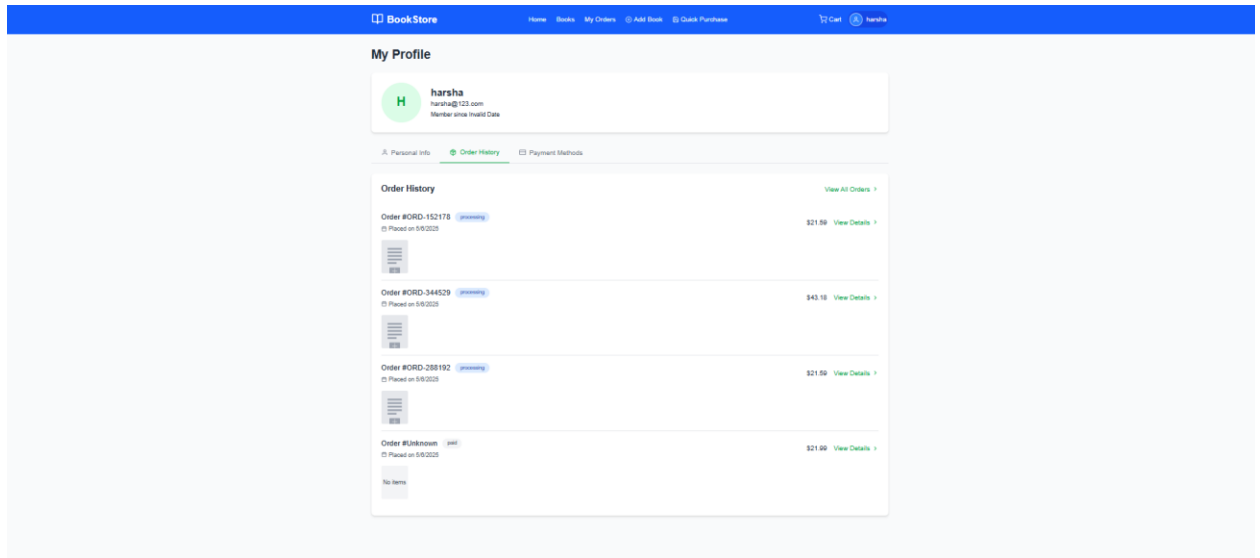
- Profile view screenshot

The screenshot displays the 'My Profile' page of a 'BookStore' application. The page features a blue header with navigation links: Home, Books, My Orders, Add Book, and Quick Purchase. A user profile card at the top shows a green circular avatar with the letter 'H', the name 'harsha', email 'harsha@123.com', and 'Member since Invalid Date'. Below the card are three tabs: 'Personal Info' (active), 'Order History', and 'Payment Methods'. The 'Personal Information' section contains input fields for Full Name (harsha), Email Address (harsha@123.com), and Phone Number (+1 (888) 123-4567), followed by a green 'Save Changes' button. The 'Change Password' section includes fields for Current Password, New Password, and Confirm New Password, with a green 'Update Password' button at the bottom.

- Edit profile interface screenshot

This screenshot shows the 'My Profile' page with the edit interface. The layout is identical to the previous screenshot, but the input fields in the 'Personal Information' section are now active, showing the current values: 'harsha' for Full Name, 'harsha@123.com' for Email Address, and '+1 (888) 123-4567' for Phone Number. The 'Save Changes' button remains green. The 'Change Password' section also remains the same, with the 'Update Password' button at the bottom.

- Order history view screenshot



- Related implementation code screenshots

The first screenshot shows the implementation of the `OrderDetailPage` component. It uses `useState` and `useEffect` to manage the order state and fetch details. The component is part of a Next.js application with a file explorer on the left and a terminal at the bottom.

```
1 'use client';
2
3 import { useState, useEffect } from 'react';
4 import { useRouter, useParams } from 'next/navigation';
5 import { motion } from 'framer-motion';
6 import { FiArrowLeft, FiPackage, FiCreditCard, FiMapPin, FiCalendar, FiClock, FiCheckCircle, FiDownload, FiTruck, FiX } from 'react-icons/fi';
7 import { useAuth } from '../context/AuthContext';
8 import { ordersAPI } from '../services/api';
9 import toast from 'react-hot-toast';
10 import Link from 'next/link';
11 import AnimatedButton from '../components/AnimatedButton';
12
13 export default function OrderDetailPage() {
14   const [order, setOrder] = useState<any>(null);
15   const [loading, setLoading] = useState(true);
16   const { isAuthenticated } = useAuth();
17   const router = useRouter();
18   const params = useParams();
19   const orderId = params?.id as string;
20
21   // Load order details
22   useEffect(() => {
23     async function loadOrderDetails() {
24       if (!orderId) return;
25
26       try {
27         setLoading(true);
28         const response = await ordersAPI.getOrderById(orderId);
29         setOrder(response.order);
30       } catch {
31         // Error handling
32       }
33     }
34     loadOrderDetails();
35   }, [orderId]);
36 }
```

The second screenshot shows the TypeScript interfaces for the data. It defines `OrderItem` and `Order` interfaces.

```
1 'use client';
2
3 import { useState, useEffect } from 'react';
4 import { useRouter } from 'next/navigation';
5 import { motion, AnimatePresence } from 'framer-motion';
6 import { FiPackage, FiCreditCard, FiShoppingBag, FiCalendar, FiChevronRight, FiClock, FiCheckCircle } from 'react-icons/fi';
7 import { useAuth } from '../context/AuthContext';
8 import { ordersAPI } from '../services/api';
9 import toast from 'react-hot-toast';
10 import Link from 'next/link';
11
12 // Define TypeScript interfaces for our data
13 interface OrderItem {
14   _id: string;
15   bookId: string;
16   book: {
17     _id: string;
18     title: string;
19     authors: string[];
20     imageLinks: {
21       thumbnail: string;
22     };
23   };
24   quantity: number;
25   price: number;
26 }
27
28 interface Order {
29   _id: string;
30 }
```

